

Suppose you write a class named `SomeClass`, then create an object from that class, like this:

---

```
oSomeObject = SomeClass(<optional arguments>)
```

---

The object `oSomeObject` contains a set of all the instance variables defined in the class. Every method of the `SomeClass` class has a definition that looks like this:

---

```
def someMethod(self, <any other parameters>):
```

---

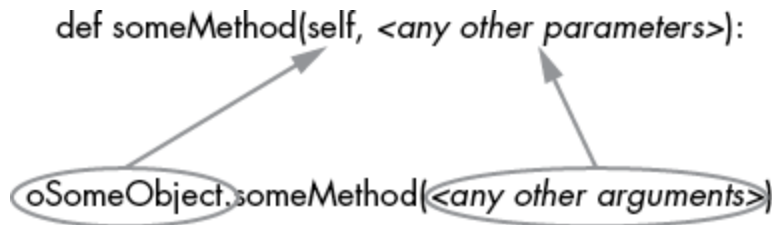
And here is the general form of a call to such a method:

---

```
oSomeObject.someMethod(<any other arguments>)
```

---

As we know, Python rearranges the arguments in a call to a method, so that the object is passed in as the first argument. That value is received in the first parameter of the method and is put into the variable `self` ([Figure 3-4](#)).



[Figure 3-4](#): How Python rearranges arguments in a call to a method

Therefore, whenever a method is called, `self` will be set to the object in the call. This means that the code of a method can operate on the instance variables of *any* object instantiated from the class. It does so using the form:

---

```
self.<instanceVariableName>
```

---

This essentially says to use the object referred to by `self` and access the instance variable specified by `<instanceVariableName>`. Since every method uses `self` as the first parameter, every method in a class uses this same approach.